4. Documento de Arquitetura

4.1 Arquitetura adotada

O sistema adota uma arquitetura em camadas, em que cada camada só conhece a camada imediatamente abaixo dela:

Camada	Responsabilidade	Componentes
Apresentação	Receber os dados do usuário e exibir resultados	Menu de terminal (CLI) e página web (index.html)
Aplicação	Expor e orquestrar os casos de uso: cadastrar, consultar, alterar, excluir, gerar folha individual e resumo consolidado	SistemaFolha e API REST em Flask
Domínio	Regras de negócio (RN001 a RN008) e cálculo da remuneração	Colaborador (abstrata), ColaboradorPadrao, ColaboradorComissionado, ColaboradorProducao, ValidacaoError
Persistência	Gravar e restaurar os colaboradores	Repositório SQLite

As duas interfaces usam o mesmo SistemaFolha, então uma regra de negócio é implementada uma única vez e vale para o terminal e para a web. A camada de domínio não conhece banco de dados nem interface, o que permite testá-la isoladamente.

A extensibilidade exigida pelo RNF004 vem do princípio aberto/fechado: um novo tipo de remuneração é uma nova subclasse de Colaborador que implementa calcular_salario_bruto(), sem alterar os cálculos existentes.

Situação do código herdado do T1: o domínio e o menu de terminal existem; a página web já chama a API em http://127.0.0.1:5000/api/colaboradores, mas o servidor Flask ainda não existe; a persistência atual é em arquivo JSON e passa para SQLite na A3.

4.2 Tecnologias selecionadas

Camada	Tecnologia
Front-end	HTML5, CSS e JavaScript (Fetch API), sem framework
Back-end	Python 3.12 ou superior, Flask 3
Banco	SQLite 3, pelo módulo sqlite3 da biblioteca padrão do Python
Testes	pytest, pytest-cov e o cliente de testes do Flask
Qualidade de código	ruff (verificação de estilo e formatação)
Versionamento e gestão	Git, GitHub (Issues e Projects)
Modelagem	PlantUML

4.3 Justificativas técnicas

- **Python:** a equipe já domina a linguagem, o T1 foi escrito nela e a orientação a objetos com classes abstratas (abc) representa bem a hierarquia de colaboradores.

- **Flask:** a API precisa de poucas rotas. Um microframework atende com pouco código e tem cliente de testes próprio, usado nos testes de integração.
- **SQLite:** já vem com o Python, não exige servidor nem instalação na máquina de quem corrige, guarda tudo num arquivo só e aceita SQL padrão, o que permite versionar o script de criação do banco na pasta 06_Banco_Dados.
- **HTML e JavaScript sem framework:** a página roda direto no navegador, sem etapa de compilação.
- **pytest e pytest-cov:** padrão de mercado em Python, com fixtures para isolar cada teste e relatório de cobertura para medir a meta de 80%.
- **GitHub Issues e Projects:** concentram tarefas, defeitos e cronograma no mesmo lugar do código, o que dá rastreabilidade entre requisito, issue, commit e Pull Request.

4.4 Organização dos componentes

Estrutura prevista para 05_Codigo_Fonte na A3:

Arquivo	Conteúdo	Depende de
dominio.py	Colaborador, subclasses, ValidacaoError, cálculos	nada
repositorio.py	Gravação, leitura e exclusão no SQLite	dominio.py
sistema.py	SistemaFolha: casos de uso, folha individual e resumo consolidado	dominio.py, repositorio.py
cli.py	Menu de terminal	sistema.py
api.py	Rotas Flask	sistema.py
web/index.html	Página web	api.py (via HTTP)

Rotas da API:

Método e rota	Caso de uso
GET /api/colaboradores	Consultar colaboradores e resumo (RF005, RF008)
GET /api/colaboradores/<matricula>	Folha individual (RF006, RF007)
POST /api/colaboradores	Cadastrar (RF001 a RF004)
PUT /api/colaboradores/<matricula>	Alterar (RF009)
DELETE /api/colaboradores/<matricula>	Excluir (RF009)

Erros de validação (ValidacaoError) viram resposta HTTP 400 com a mensagem da regra violada; matrícula inexistente vira 404.

Os relatórios passam a ser montados em SistemaFolha como dados (listas e totais) e formatados por cada interface. Hoje gerar_folha_individual e emitir_resumo_folha imprimem direto no terminal, o que impede a web de reaproveitá-los.

4.5 Estratégia de persistência

Os colaboradores ficam numa única tabela SQLite, com uma coluna tipo que indica a categoria (herança em tabela única). As regras de negócio também são garantidas pelo banco, como segunda barreira:

Coluna	Tipo	Restrição	Regra
matricula	TEXT	chave primária	RN001
nome	TEXT	não nulo e não vazio	RN002
tipo	TEXT	Padrao, Comissionado ou Producao	RF002 a RF004
salario_base	REAL	maior ou igual a zero	RN003
vendas	REAL	maior ou igual a zero	RN004
comissao_pct	REAL	maior ou igual a zero	RN005
qtd_produzida	INTEGER	maior ou igual a zero	RN006
valor_unidade	REAL	maior ou igual a zero	RN007

- O salário bruto, o INSS, o IRRF e o líquido não são gravados: são calculados a cada consulta (RN008), então nunca ficam desatualizados.
- O script 06_Banco_Dados/schema.sql cria a tabela, e o sistema o executa na inicialização se a tabela ainda não existir.
- Toda consulta usa parâmetros (?) em vez de montar o SQL com texto, o que impede injeção de SQL.
- Cada operação de escrita é confirmada numa transação; se falhar, nada é gravado pela metade.
- Na carga, um registro inválido é informado com a matrícula e o motivo, e os demais continuam sendo carregados. No T1, um registro inválido interrompe a carga de todos os seguintes, com um aviso genérico que não diz qual registro falhou (folhadepagamento.py, linhas 242 e 243).
- Os testes usam um banco temporário próprio, nunca o arquivo de uso real.

8. Planejamento do Projeto

8.1 Estrutura analítica do projeto (EAP)

Código	Pacote de trabalho	Entregáveis	Responsável
1	Projeto Sistema de Folha de Pagamento	Sistema e artefatos da A1 e da A3	Equipe
1.1	Gestão do projeto	EAP, cronograma, distribuição de responsabilidades, atas de revisão	Iago e Ana
1.2	Requisitos	Documento de Visão, ERS, matriz de rastreabilidade	Ana e Mariana
1.3	Análise e projeto	Diagramas UML, Documento de Arquitetura	Mariana e João
1.4	Garantia da qualidade	PGQ, Plano de Testes, casos de teste, registro de defeitos, métricas	Iago, João, Mariana e Ana
1.5	Construção	Código-fonte, banco de dados, API, interfaces de terminal e web	João
1.6	Testes	Testes unitários, de integração e de aceitação, relatório de testes	João e Ana
1.7	Documentação	README, Manual do Usuário, Manual Técnico	João e Mariana
1.8	Homologação e entrega	Termo de Homologação, Relatório Final de Qualidade, apresentações	Ana, Mariana e Iago

8.2 Cronograma

Etapa	Início	Fim	Responsável
Visão, ERS e planejamento	07/10	13/10	Ana, Mariana e Iago
UML, Arquitetura, PGQ e Plano de Testes	07/10	15/10	Mariana, João e Iago
Casos de teste	13/10	16/10	Mariana e Ana
Revisão cruzada da A1 (ata)	19/10	19/10	Iago
Entrega e apresentação da A1	20/10	20/10	Equipe
Correções da avaliação da A1 (versão 1.1)	30/10	30/10	João e Iago
Implementação (RF001 a RF010, SQLite, API)	21/10	01/11	João
Testes unitários e de integração	02/11	08/11	João
Testes de aceitação e registro de defeitos	09/11	22/11	Ana e Iago
Manuais, métricas e relatórios	09/11	28/11	Mariana, João e Iago
Termo de homologação e apresentação	23/11	01/12	Ana e Iago
Entrega da A3	01/12	01/12	Equipe

O cronograma detalhado, item a item, está no GitHub Projects do repositório: github.com/users/johnrosik/projects/3.

8.3 Distribuição de responsabilidades

Integrante	Papel	A1	A3
João Vitor Rosiak de O. Baltezan	Desenvolvimento	Documento de Arquitetura, partes técnicas do PGQ e do Plano de Testes, repositório e README	Sistema, banco, testes unitários e de integração, Manual Técnico
Mariana F. Hammel da Silva	Requisitos e modelagem	ERS, diagramas UML, casos de teste CT001 a CT010	Relatório de Revisão Técnica, Manual do Usuário, Relatório Final de Qualidade
Iago Ferst Tubiana	Gestão e qualidade	PGQ, Plano de Testes, planejamento, ata da revisão cruzada	Registro de defeitos, métricas, relatório de testes, atas, apresentação
Ana Clara S. F. Gomes	Requisitos, testes e apresentação	Documento de Visão, casos de teste CT011 a CT020, montagem da entrega	Testes de aceitação, Termo de Homologação, apresentação, montagem da entrega

Cada artefato tem uma issue no repositório, com dono, apoio, prazo e checklist.

9. Repositório do Projeto

9.1 README

O README do repositório apresenta a equipe e os papéis, onde fica cada parte do trabalho, a estrutura de pastas, como executar o sistema, como contribuir e as datas de entrega. Endereço: github.com/johnrosik/folha-pagamento-gqs.

9.2 Histórico de commits

Toda mudança entra por Pull Request ligado a uma issue, e a mesclagem preserva os commits de cada integrante (5.2). O histórico completo está no próprio repositório.

A partir da versão 1.1, cada artefato da A1 também fica na pasta do seu tipo, em PDF: 01_Requisitos (Visão e ERS), 02_Analise (Modelagem UML, com o código PlantUML e as imagens), 03_Projeto (Arquitetura e Planejamento), 04_Testes (Plano de Testes e Casos de Teste) e 10_Qualidade (PGQ e ata de revisão). O documento completo continua em 11_Apresentacao.

9.3 Estrutura inicial do projeto

Pasta	Conteúdo
01_Requisitos	Documento de Visão, ERS, matriz de rastreabilidade
02_Analise	Diagramas UML (PlantUML e imagens)
03_Projeto	Documento de Arquitetura
04_Testes	Plano de Testes, casos de teste, relatórios de execução e de cobertura
05_Codigo_Fonte	Código do sistema (hoje, o código do T1)
06_Banco_Dados	Script de criação do banco SQLite
07_Executavel	Instruções e script para executar o sistema
08_Manuais	Manual do Usuário e Manual Técnico
09_Homologacao	Testes de aceitação e Termo de Homologação
10_Qualidade	PGQ, registro de defeitos, métricas, atas e Relatório Final de Qualidade
11_Apresentacao	Planejamento do projeto e slides